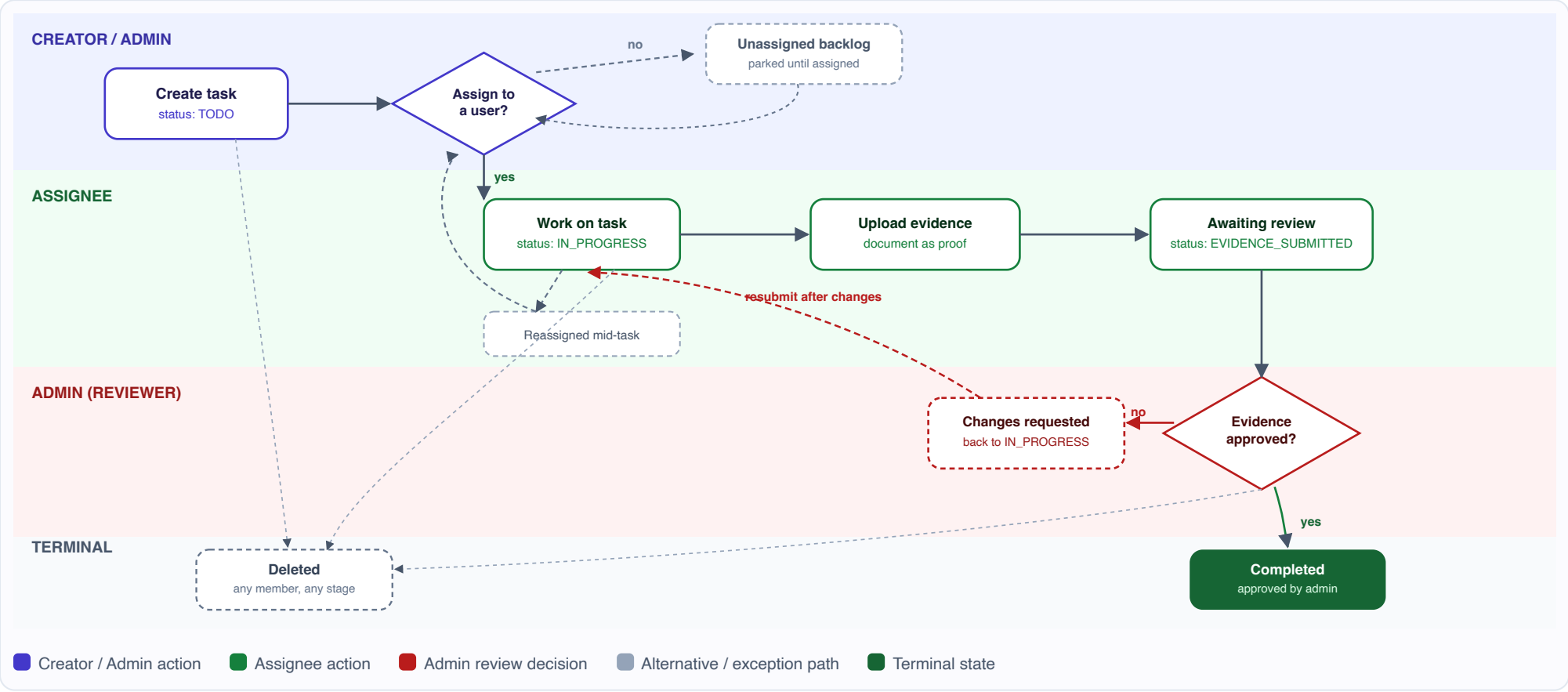
INTERNAL TASK PORTAL

Part 2 — Design the Next Iteration

Process flow map and sprint implementation plan for status/assignee filtering, a task details screen, evidence upload, and admin review.

DELIVERABLE 1 OF 2

Process Flow Map — Task Lifecycle



Happy path: Create → Assign → Work (In Progress) → Upload evidence → Admin approves → Completed.

Rejected evidence: admin can request changes instead of approving; task returns to In Progress for the assignee to resubmit. Approving also requires evidence to actually be attached, not just the status label.

Deletion, not cancellation: there's no soft "Cancelled" status. Any member can permanently delete a task from any active state — a hard delete (evidence goes with it), not a status, with nothing kept for audit.

Read access: not shown — per the Part 1 decision, every authenticated user can view every task; only the assignee uploads evidence, only an admin approves.

Unassigned backlog: created without an assignee, parked until anyone assigns it (no owner/admin restriction), then re-enters at "Work on task." Evidence can't be uploaded with no assignee, since only the assignee may upload.

Reassignment: any authenticated member can reassign a task at any stage — a plain field edit, not a gated step, and not limited to its creator/admin. Existing evidence and approval state aren't cleared by it.

Filtering isn't shown here on purpose: filtering by status/assignee (Sprint 1) is a list/query capability, not a state a task passes through, so it applies uniformly across every state below rather than being a step of its own.

DELIVERABLE 2 OF 2

Sprint Implementation Plan

Broken into four sequential sprints, each shippable and demoable on its own, followed by an optional polish sprint. Sequencing follows the dependency chain: assignment must exist before it can be filtered on; the task details screen must exist before evidence can be uploaded to it; evidence must exist before it can be reviewed.

SPRINT	GOAL	PRIMARY REQUIREMENT(S) COVERED
S1	Assignment model + filtering by status and assigned person	Filter tasks by status and assigned person
S2	Dedicated task details screen	Selecting a task opens a details screen
S3	Evidence upload on the details screen	Assignee uploads a document as evidence of completion
S4	Admin review and approval	Admin reviews evidence and approves completion
S5	Polish, audit trail, hardening (stretch)	Not explicitly required — reduces launch risk

Cross-cutting assumption: "assigned person" is a new concept, distinct from the Part 1 task *owner* (creator). The owner retains edit/delete rights; the assignee is the person expected to do the work and submit evidence. A task's owner and assignee may be the same user or different users. This split is what makes admin review meaningful — the admin is checking the assignee's work, which may have been created and delegated by someone else.

Sprint 1 — Assignment model & filtering

Depends on: nothing (builds directly on the Part 1 schema). Unlocks: S2, S3, S4.

#	TASK	TYPE	ACCEPTANCE CRITERIA
1.1	Add `assigneeId` (nullable FK to User) to Task; migration + seed data update.	DATA	• Existing tasks migrate with `assigneeId = null` (unassigned backlog), no data loss.
1.2	Assign/reassign a user to a task.	BE FE	• **As built:** no owner/admin restriction, matching this app's no-ownership-gating model elsewhere; reassigning doesn't clear existing evidence/approval state.
1.3	Filter controls: status and assigned person, reflected in the URL query string.	FE	• Filters are shareable/bookmarkable via URL (e.g. `/tasks?status=IN_PROGRESS&assignee=<id>`). "Unassigned" is a selectable value. • **As built:** single-select dropdowns, not multi-select or searchable — this portal's team size doesn't need either.
1.4	Server-side filtering by `status`/`assigneeId`.	BE	• Filtering happens server-side, not client-side, to stay correct as the task list grows. • **As built:** not `GET /api/tasks` query params as planned — the list page reads `searchParams` and queries Prisma directly, since that data's needed for the render anyway. The endpoint still exists, unfiltered, for client-side revalidation only.

Sprint 2 — Task details screen

Depends on: S1 (needs assignee data to display). Unlocks: S3, S4.

#	TASK	TYPE	ACCEPTANCE CRITERIA
2.1	New route `/tasks/[id]` rendering full task detail (title, description, status, owner, assignee, timestamps).	FE	• Clicking a task card in the list navigates to its details screen instead of opening the inline edit dialog. • Unauthenticated or unauthorized access redirects/errors the same way the list does.
2.2	Move edit/delete/status-change controls from the list card onto the details screen.	FE	• List card becomes a lighter-weight summary (title, status badge, assignee); management actions live on the details screen only. • **As built, beyond this sprint's original scope:** the board also gained drag-and-drop between status columns as a faster path to the same status change the details screen's Edit dialog makes — still the same PATCH endpoint and authorization, just a second entry point.
2.3	Task detail data available to the details screen.	BE	• Reuses the existing role authorization check from PATCH/DELETE (no ownership gating — see 1.2). • **As built:** not a `GET /api/tasks/[id]` route as planned — like the list page, `/tasks/[id]` is a Server Component that reads the task directly via Prisma, so there's no separate read endpoint for a single task either.

Continued on next page — Sprints 3–5, key dependencies, and assumptions.

DELIVERABLE 2 OF 2 (CONTINUED)

Sprint Implementation Plan — Sprints 3–5

Sprint 3 — Evidence upload

Depends on: S2 (needs a details screen to upload from). Unlocks: S4.

#	TASK	TYPE	ACCEPTANCE CRITERIA
3.1	Choose and provision file storage (e.g. S3-compatible bucket or managed blob storage) and wire up server-side upload credentials.	INFRA	• Decision recorded: local disk is unsuitable beyond a demo (ephemeral on most hosts) — needs a real object store before this sprint starts coding against it.
3.2	Add `TaskEvidence` table (taskId, uploadedById, fileName, `blobUrl`, mimeType, `fileSize`, uploadedAt).	DATA	• One-to-many from Task, so re-submission keeps history instead of overwriting. • **As built:** stored as a *private* blob — Vercel rejects any request for its URL that isn't authenticated with this app's own storage token, so files are only ever readable through a dedicated download route (see 4.1), never a raw public link.
3.3	Upload control on the details screen, visible only to the assigned user.	FE	• **As built:** enabled for the assignee whenever the task isn't already Done, not restricted to In Progress/post-changes-requested specifically — simpler to reason about, and status alone was going to allow the same set of "not finished yet" states anyway. • Client-side file type/size validation with a clear error message before upload.
3.4	`POST /api/tasks/[id]/evidence`: validates file server-side, stores it, sets status to Evidence Submitted.	BE	• Rejects uploads from anyone other than the current assignee (403), and rejects disallowed file types/oversized files (400). • Server-side re-validation of file type/size — never trust the client check alone.

Sprint 4 — Admin review & approval

Depends on: S3 (needs evidence to review). Unlocks: nothing further in this iteration.

#	TASK	TYPE	ACCEPTANCE CRITERIA
4.1	Review panel on the details screen (visible to ADMIN role only): evidence preview/download, Approve and Request Changes actions.	FE	• Non-admin users never see review controls, even via direct URL (server-checked, not just hidden in the UI). • **As built:** "download" goes through a session-gated route, not a link to the file's own URL, since evidence is stored privately (see 3.2) — any authenticated member can open it, matching who can already view the task itself.
4.2	`POST /api/tasks/[id]/approve` and `POST /api/tasks/[id]/request-changes`, both admin-only.	BE	• Approve sets status Done and records `approvedById`/`approvedAt`. • Request Changes sets status back to In Progress and (optionally) stores a reviewer comment. • Both reject non-admin callers with 403, reusing the Role field already in the Part 1 schema. • **Added beyond the original plan:** Approve additionally requires at least one evidence record to actually exist, not just the status label saying so — closes a gap where a task could otherwise reach "Pending review" with nothing attached and still get approved. • **Added beyond the original plan:** Done is unreachable through any other path at all — including the general task-update endpoint used for every other field, for every role including admin. It's the one status that can only ever be set by this endpoint.

#	TASK	TYPE	ACCEPTANCE CRITERIA
4.3	Reviewer comment field on Request Changes, shown to the assignee.	FE BE	• Assignee sees why the evidence was rejected without needing to ask the admin directly.

Sprint 5 — Polish & hardening (stretch, not required by the brief)

#	TASK	TYPE	ACCEPTANCE CRITERIA / RATIONALE
5.1	Activity/audit log (assigned, status changes, evidence submitted, reviewed).	DATA	Still not built. Not required by the brief, but review workflows typically need traceability of who did what and when.
5.2	Notifications (in-app or email) on assignment, submission, and review outcome.	BE	Still not built. Keeps assignees/admins from having to poll the portal for state changes.
5.3	Malware/content scanning on uploaded evidence.	INFRA	Still not built. Required before production launch if evidence files are user-uploaded and later opened by admins.

Built later, outside this document's scope: a "Users" admin screen (view every account, promote/demote the `ADMIN` role, with an admin blocked from changing their own role) shipped after this plan, in response to a direct request rather than the original four Part 2 requirements. It isn't sprint-planned here since it's a different feature area (account/permission management, not the task lifecycle this document scopes) — noted for completeness rather than retrofitted into the plan above.

Key Dependencies, Assumptions & Open Questions

Key dependencies (why this order)

DEPENDS ON	BECAUSE
S1 → S2	Details screen displays the assignee, which doesn't exist before S1.
S2 → S3	Evidence is uploaded from the details screen, which doesn't exist before S2.
S3 → S4	Admin can't review evidence that doesn't exist yet.
S3.1 (storage) → S3.2–3.4	Data model and upload code are shaped by the storage choice (direct-to-bucket vs. via server) — decide first.

Assumptions made for this plan

ASSUMPTION	WHY IT'S REASONABLE
One evidence document per submission is enough for v1; the schema still allows many for history.	Matches the brief's "upload a document" (singular); multi-file is a small, additive change later.
Only one admin approval is required to complete a task (no multi-approver workflow).	Brief says "an admin user," singular; multi-approver would be a distinct, larger feature.
Rejected evidence returns the task to In Progress rather than a separate "Rejected" terminal state.	Keeps the state machine small; the task is still actionable work, not truly rejected.
Reassignment and a way to remove a no-longer-needed task are both required even though not explicitly requested.	Any real assignment workflow needs an escape hatch for "wrong person" and "no longer needed" — flagged per the brief's "include alternative paths where you believe they are needed." **As built:** the latter is permanent deletion, not a soft "Cancelled" status — see the flow map's note on this.

Open questions, as resolved by what was actually built

QUESTION	RESOLVED AS
What file types/sizes count as valid "evidence"?	PDF, Word (.doc/.docx), PNG/JPEG/WebP; 10MB max, enforced server-side regardless of what the file picker already narrows to client-side.
Can a task have more than one assignee, or is it always exactly one?	Exactly one — a nullable FK, not a join table, per S1.1 as planned.
Should rejected evidence be deleted, or retained for audit alongside the approved one?	Retained — a rejection only sets the reviewer note and moves the status back; the evidence record and file are never removed by a rejection.
Does the assignee get notified automatically, or is polling the portal acceptable for v1?	Polling — S5.2 (notifications) was never pulled forward; still not built.

Out of scope for this iteration

Multi-approver review chains, SLA/due-date tracking, task comments/threads, and email notifications are all reasonable next steps but are not implied by the four stated requirements — listed here so they're consciously deferred rather than silently missing.